

DBMS Lab 6 - Assignment Completion – Triggers , Procedures & Functions

University Fest Management System

Name : Adishree Gupta

SRN : PES1UG23CS024

SECTION : A

TASK 1: TRIGGERS

Trigger 1: Decrease Quantity Purchase

Screenshot 1 - :

CODE

```
SELECT * FROM stall_items WHERE stall_id = 'S1' AND item_name = 'Veggie Wrap';
```

```
mysql> SELECT * FROM stall_items WHERE stall_id = 'S1' AND item_name = 'Veggie Wrap';
+-----+-----+-----+-----+
| stall_id | item_name | price_per_unit | total_quantity |
+-----+-----+-----+-----+
| S1      | Veggie Wrap | 399.00 | 47 |
+-----+-----+-----+-----+
1 row in set (0.08 sec)
```

Screenshot 2 - COMMAND:

CODE

```
DELIMITER $$
```

```
CREATE TRIGGER decrease_quantity_ _purchase
```

```
INSERT ON purchased
```

```
FOR EACH ROW
```

```
BEGIN
```

```
UPDATE stall_items
```

```
SET total_quantity = total_quantity - NEW.quantity
```

```
WHERE stall_id = NEW.stall_id AND item_name = NEW.item_name;
```

```
END$$
```

```
DELIMITER ;
```

```
mysql> DROP TRIGGER IF EXISTS decrease_quantity_after_purchase;
Query OK, 0 rows affected, 1 warning (0.02 sec)

mysql> DELIMITER $$
mysql> CREATE TRIGGER decrease_quantity_after_purchase
  -> AFTER INSERT ON purchased
  -> FOR EACH ROW
  -> BEGIN
  ->     UPDATE stall_items
  ->     SET total_quantity = total_quantity - NEW.quantity
  ->     WHERE stall_id = NEW.stall_id
  ->     AND item_name = NEW.item_name;
  -> END$$
Query OK, 0 rows affected (0.06 sec)
```

Screenshot 3 - :

CODE

INSERT INTO purchased VALUES

('P1020', 'S1', 'Veggie Wrap', '2023-04-18 14:30:00', 3);

SELECT * FROM stall_items WHERE stall_id = 'S1' AND item_name = 'Veggie Wrap';

```
mysql> INSERT INTO purchased VALUES
  -> ('P1020', 'S1', 'Veggie Wrap', '2023-04-18 14:30:00', 3);
Query OK, 1 row affected (0.05 sec)

mysql> SELECT * FROM stall_items WHERE stall_id = 'S1' AND item_name = 'Veggie Wrap';
+-----+-----+-----+-----+
| stall_id | item_name | price_per_unit | total_quantity |
+-----+-----+-----+-----+
| S1      | Veggie Wrap | 399.00 | 41 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Trigger 2: Prevent Bulk Purchase (> 5 units)

Screenshot 1 - :

CODE

SELECT * FROM purchased WHERE SRN = 'P1015' ORDER BY timestamp DESC LIMIT 5;

```
mysql> SELECT * FROM purchased WHERE SRN = 'P1015' ORDER BY timestamp DESC LIMIT 5;
Empty set (0.00 sec)
```

Screenshot 2 - COMMAND:

CODE

DELIMITER \$\$

CREATE TRIGGER prevent_bulk_purchase

INSERT ON purchased

FOR EACH ROW

```

BEGIN
  IF NEW.quantity > 5 THEN
    SIGNAL CODESTATE '45000'
    SET MESSAGE_TEXT = 'Error: Cannot purchase more than 5 units in one transaction';
  END IF;
END$$
DELIMITER ;

```

```

mysql> DROP TRIGGER IF EXISTS prevent_bulk_purchase;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DELIMITER $$
mysql> CREATE TRIGGER prevent_bulk_purchase
  -> BEFORE INSERT ON purchased
  -> FOR EACH ROW
  -> BEGIN
  ->     IF NEW.quantity > 5 THEN
  ->         SIGNAL SQLSTATE '45000'
  ->         SET MESSAGE_TEXT = 'Error: Cannot purchase more than 5 units
of a single item in one transaction';
  ->     END IF;
  -> END$$
Query OK, 0 rows affected (0.01 sec)

```

Screenshot 3 - :

CODE

-- Test with valid quantity (≤5)

```

INSERT INTO purchased VALUES
('P1015', 'S2', 'Veggie Wrap', '2023-04-18 15:00:00', 4);

```

```

SELECT * FROM purchased WHERE SRN = 'P1015' ORDER BY timestamp DESC LIMIT 5;

```

```

mysql> INSERT INTO purchased VALUES
  -> ('P1015', 'S2', 'Veggie Wrap', '2023-04-18 15:00:00', 4);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM purchased WHERE SRN = 'P1015' ORDER BY timestamp DESC L
IMIT 5;
+-----+-----+-----+-----+-----+
| SRN   | stall_id | item_name  | timestamp           | quantity |
+-----+-----+-----+-----+-----+
| P1015 | S2       | Veggie Wrap | 2023-04-18 15:00:00 | 4        |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

CODE

```

INSERT INTO purchased VALUES
('P1015', 'S2', 'Veggie Wrap', '2023-04-18 16:00:00', 6);

```

```
mysql> INSERT INTO purchased VALUES
-> ('P1015', 'S2', 'Veggie Wrap', '2023-04-18 16:00:00', 6);
ERROR 1644 (45000): Error: Cannot purchase more than 5 units of a single item in one transaction
mysql>
```

TASK 2: PROCEDURES

Procedure 1: GetStallSales

Screenshot 1 - :

CODE

```
SELECT s.stall_id, s.item_name, s.price_per_unit,
       COALESCE(SUM(p.quantity), 0) as total_sold
FROM stall_items s
LEFT JOIN purchased p ON s.stall_id = p.stall_id AND s.item_name = p.item_name
WHERE s.stall_id = 'S1'
GROUP BY s.stall_id, s.item_name, s.price_per_unit;
```

```
mysql> SELECT s.stall_id, s.item_name, s.price_per_unit,
->         COALESCE(SUM(p.quantity), 0) as total_sold
-> FROM stall_items s
-> LEFT JOIN purchased p ON s.stall_id = p.stall_id AND s.item_name = p.
item_name
-> WHERE s.stall_id = 'S1'
-> GROUP BY s.stall_id, s.item_name, s.price_per_unit;
```

stall_id	item_name	price_per_unit	total_sold
S1	Caprese Salad	249.00	3
S1	Classic Caesar Salad	349.50	6
S1	Margherita Pizza	350.00	1
S1	Mushroom Risotto	359.00	3
S1	Spinach and Feta Omelette	259.00	2
S1	Vegetable Pad Thai	329.00	0
S1	Vegetable Stir-Fry	299.00	0
S1	Veggie Wrap	399.00	8

8 rows in set (0.04 sec)

Screenshot 2 - COMMAND:

CODE

```
DELIMITER $$
CREATE PROCEDURE GetStallSales(IN input_stall_id VARCHAR(5))
BEGIN
    SELECT p.stall_id, SUM(s.price_per_unit * p.quantity) AS total_revenue
    FROM purchased p
    JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s.item_name
```

```

WHERE p.stall_id = input_stall_id
GROUP BY p.stall_id;

SELECT p.stall_id, p.item_name, s.price_per_unit,
       SUM(p.quantity) AS total_quantity_sold,
       SUM(s.price_per_unit * p.quantity) AS item_revenue
FROM purchased p
JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s.item_name
WHERE p.stall_id = input_stall_id
GROUP BY p.stall_id, p.item_name, s.price_per_unit;
END$$
DELIMITER ;

```

```

mysql> DELIMITER $$
mysql> CREATE PROCEDURE GetStallSales(IN input_stall_id VARCHAR(5))
-> BEGIN
->     SELECT p.stall_id, SUM(s.price_per_unit * p.quantity) AS total_re
venue
->     FROM purchased p
->     JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s
.item_name
->     WHERE p.stall_id = input_stall_id
->     GROUP BY p.stall_id;
->
->     SELECT p.stall_id, p.item_name, s.price_per_unit,
->            SUM(p.quantity) AS total_quantity_sold,
->            SUM(s.price_per_unit * p.quantity) AS item_revenue
->     FROM purchased p
->     JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s
.item_name
->     WHERE p.stall_id = input_stall_id
->     GROUP BY p.stall_id, p.item_name, s.price_per_unit;
-> END$$
Query OK, 0 rows affected (0.09 sec)

```

Screenshot 3 - :

CODE

```
CALL GetStallSales('S1');
```

```
mysql> CALL GetStallSales('S1');
+-----+-----+
| stall_id | total_revenue |
+-----+-----+
| S1      | 7981.00      |
+-----+-----+
1 row in set (0.03 sec)

+-----+-----+-----+-----+
--+-----+
| stall_id | item_name          | price_per_unit | total_quantity_sol
d | item_revenue |
+-----+-----+-----+-----+
--+-----+
| S1      | Mushroom Risotto   | 359.00         |
3 | 1077.00         |
| S1      | Veggie Wrap        | 399.00         |
8 | 3192.00         |
| S1      | Caprese Salad      | 249.00         |
3 | 747.00          |
| S1      | Classic Caesar Salad | 349.50         |
6 | 2097.00         |
| S1      | Margherita Pizza    | 350.00         |
1 | 350.00          |
| S1      | Spinach and Feta Omelette | 259.00         |
2 | 518.00          |
+-----+-----+-----+-----+
--+-----+
6 rows in set (0.03 sec)

Query OK, 0 rows affected (0.05 sec)
```

Procedure 2: RegisterParticipant

Screenshot 1 - :

CODE

```
SELECT * FROM registration WHERE event_id = 'E15' ORDER BY SRN DESC LIMIT 5;
SELECT COUNT(*) as registration_count FROM registration WHERE event_id = 'E15';
```

```
mysql> SELECT * FROM registration WHERE event_id = 'E15' ORDER BY SRN DESC LIMIT 5;
```

event_id	SRN	registration_id
E15	P1025	R75
E15	P1024	R74
E15	P1023	R73
E15	P1022	R72
E15	P1021	R71

```
5 rows in set (0.11 sec)
```

```
mysql> SELECT COUNT(*) as registration_count FROM registration WHERE event_id = 'E15';
```

registration_count
5

```
1 row in set (0.00 sec)
```

Screenshot 2 - COMMAND:

CODE

```
DELIMITER $$
```

```
CREATE PROCEDURE RegisterParticipant(
```

```
    IN input_event_id VARCHAR(5),
```

```
    IN input_SRN VARCHAR(10),
```

```
    IN input_registration_id VARCHAR(5)
```

```
)
```

```
BEGIN
```

```
    DECLARE registration_exists INT;
```

```
    SELECT COUNT(*) INTO registration_exists
```

```
    FROM registration WHERE event_id = input_event_id AND SRN = input_SRN;
```

```
    IF registration_exists > 0 THEN
```

```
        SIGNAL CODESTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Error: Participant already registered for this event';
```

```
    ELSE
```

```
        INSERT INTO registration (event_id, SRN, registration_id)
```

```
        VALUES (input_event_id, input_SRN, input_registration_id);
```

```
        SELECT CONCAT('Successfully registered participant ', input_SRN) AS result;
```

```
    END IF;
```

```
END$$
```

```
DELIMITER ;
```

```

mysql> DELIMITER $$
mysql> CREATE PROCEDURE RegisterParticipant(
->     IN input_event_id VARCHAR(5),
->     IN input_SRN VARCHAR(10),
->     IN input_registration_id VARCHAR(5)
-> )
-> BEGIN
->     DECLARE registration_exists INT;
->     SELECT COUNT(*) INTO registration_exists
->     FROM registration WHERE event_id = input_event_id AND SRN = input
-> _SRN;
->
->     IF registration_exists > 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Error: Participant already registered for
-> this event';
->     ELSE
->         INSERT INTO registration (event_id, SRN, registration_id)
->         VALUES (input_event_id, input_SRN, input_registration_id);
->         SELECT CONCAT('Successfully registered participant ', input_S
-> RN) AS result;
->     END IF;
-> END$$
Query OK, 0 rows affected (0.05 sec)

```

Screenshot 3 - :

CODE

CALL RegisterParticipant('E15', 'P1016', 'R201');

SELECT * FROM registration **WHERE** event_id = 'E15' **ORDER BY** SRN **DESC LIMIT** 5;

SELECT COUNT(*) as registration_count **FROM** registration **WHERE** event_id = 'E15';

```
mysql> CALL RegisterParticipant('E15', 'P1016', 'R201');
+-----+
| result |
+-----+
| Successfully registered participant P1016 |
+-----+
1 row in set (0.07 sec)

Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> SELECT * FROM registration WHERE event_id = 'E15' ORDER BY SRN DESC L
IMIT 5;
+-----+-----+-----+
| event_id | SRN   | registration_id |
+-----+-----+-----+
| E15      | P1025 | R75             |
| E15      | P1024 | R74             |
| E15      | P1023 | R73             |
| E15      | P1022 | R72             |
| E15      | P1021 | R71             |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT COUNT(*) as registration_count FROM registration WHERE event_i
d = 'E15';
+-----+
| registration_count |
+-----+
| 6                  |
+-----+
1 row in set (0.00 sec)
```

TASK 3: FUNCTIONS

Function 1: GetEventCost

Screenshot 1 - :

CODE

```
SELECT event_id, event_name, price FROM event WHERE event_id = 'E4';
```

```
mysql> SELECT event_id, event_name, price FROM event WHERE event_id = 'E4';
+-----+-----+-----+
| event_id | event_name          | price |
+-----+-----+-----+
| E4      | Photography Contest | 850.00 |
+-----+-----+-----+
1 row in set (0.01 sec)
```

Screenshot 2 - COMMAND:

CODE

```
DELIMITER $$
```

```
CREATE FUNCTION GetEventCost(input_event_id VARCHAR(5))
```

```
RETURNS DECIMAL(10,2)
```

DETERMINISTIC

READS CODE DATA

BEGIN

DECLARE event_price DECIMAL(10,2);

SELECT price INTO event_price FROM event WHERE event_id = input_event_id;

RETURN COALESCE(event_price, 0.00);

END\$\$

DELIMITER ;

```
mysql> DELIMITER $$
mysql> CREATE FUNCTION GetEventCost(input_event_id VARCHAR(5))
  -> RETURNS DECIMAL(10,2)
  -> DETERMINISTIC
  -> READS SQL DATA
  -> BEGIN
  ->     DECLARE event_price DECIMAL(10,2);
  ->     SELECT price INTO event_price FROM event WHERE event_id = input_e
vent_id;
  ->     RETURN COALESCE(event_price, 0.00);
  -> END$$
Query OK, 0 rows affected (0.01 sec)
```

Screenshot 3 - :

CODE

SELECT GetEventCost('E4') AS event_cost;

SELECT event_id, event_name, GetEventCost(event_id) AS cost

FROM event WHERE event_id IN ('E4', 'E13', 'E20')

ORDER BY cost DESC;

```
mysql> SELECT GetEventCost('E4') AS event_cost;
+-----+
| event_cost |
+-----+
|      850.00 |
+-----+
1 row in set (0.01 sec)

mysql> SELECT event_id, event_name, GetEventCost(event_id) AS cost
  -> FROM event WHERE event_id IN ('E4', 'E13', 'E20')
  -> ORDER BY cost DESC;
+-----+-----+-----+
| event_id | event_name          | cost  |
+-----+-----+-----+
| E13      | Traditional Dance wars | 850.00 |
| E4       | Photography Contest  | 850.00 |
| E20      | Hackathon            | 650.00 |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

Function 2: GetParticipantPurchaseTotal

Screenshot 1 - :

CODE

```
SELECT p.SRN, p.item_name, p.quantity, s.price_per_unit,
       (p.quantity * s.price_per_unit) AS item_total
FROM purchased p
JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s.item_name
WHERE p.SRN = 'P1005';
```

```
mysql> SELECT p.SRN, p.item_name, p.quantity, s.price_per_unit,
->       (p.quantity * s.price_per_unit) AS item_total
-> FROM purchased p
-> JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s.item_name
-> WHERE p.SRN = 'P1005';
```

SRN	item_name	quantity	price_per_unit	item_total
P1005	Fish Tacos	4	449.00	1796.00
P1005	Chicken Noodle Soup	2	564.50	1129.00
P1005	Fish Tacos	5	481.50	2407.50
P1005	BBQ Chicken Sandwich	2	489.00	978.00
P1005	Chicken Noodle Soup	1	519.75	519.75
P1005	Fish Tacos	3	470.00	1410.00
P1005	Grilled Steak	2	573.75	1147.50
P1005	Shrimp Scampi	1	419.00	419.00
P1005	Mutton Stroganoff	1	559.00	559.00

9 rows in set (0.04 sec)

Screenshot 2 - COMMAND:

CODE

```
DELIMITER $$
CREATE FUNCTION GetParticipantPurchaseTotal(input_SRN VARCHAR(10))
RETURNS DECIMAL(10,2)
DETERMINISTIC
READS CODE DATA
BEGIN
    DECLARE total_spent DECIMAL(10,2);
    SELECT COALESCE(SUM(s.price_per_unit * p.quantity), 0.00) INTO total_spent
    FROM purchased p
    JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s.item_name
    WHERE p.SRN = input_SRN;
    RETURN total_spent;
END$$
DELIMITER ;
```

```
mysql> DELIMITER $$
mysql> CREATE FUNCTION GetParticipantPurchaseTotal(input_SRN VARCHAR(10))
    -> RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE total_spent DECIMAL(10,2);
    ->     SELECT COALESCE(SUM(s.price_per_unit * p.quantity), 0.00) INTO to
tal_spent
    ->     FROM purchased p
    ->     JOIN stall_items s ON p.stall_id = s.stall_id AND p.item_name = s
.item_name
    ->     WHERE p.SRN = input_SRN;
    ->     RETURN total_spent;
    -> END$$
Query OK, 0 rows affected (0.03 sec)
```

Screenshot 3 - :

CODE

```
SELECT GetParticipantPurchaseTotal('P1005') AS total_spent;
```

```
SELECT DISTINCT p.SRN, pt.name,
    GetParticipantPurchaseTotal(p.SRN) AS total_spent
FROM purchased p
JOIN participant pt ON p.SRN = pt.SRN
ORDER BY total_spent DESC LIMIT 5;
```

```
mysql> SELECT GetParticipantPurchaseTotal('P1005') AS total_spent;
+-----+
| total_spent |
+-----+
|    10365.75 |
+-----+
1 row in set (0.01 sec)

mysql>
mysql> SELECT DISTINCT p.SRN, pt.name,
    ->     GetParticipantPurchaseTotal(p.SRN) AS total_spent
    -> FROM purchased p
    -> JOIN participant pt ON p.SRN = pt.SRN
    -> ORDER BY total_spent DESC LIMIT 5;
+-----+-----+-----+
| SRN   | name           | total_spent |
+-----+-----+-----+
| P1005 | Jacob Jones    | 10365.75    |
| P1017 | Daniel Lopez   | 9265.00     |
| P1008 | Ava Wilson     | 4548.25     |
| P1001 | John Smith     | 3773.50     |
| P1024 | Scarlett Evans | 3651.50     |
+-----+-----+-----+
5 rows in set (0.02 sec)
```

Verification Commands

completing all tasks, verify everything is created:

CODE

-- Show all triggers

SHOW TRIGGERS FROM Fest_database;

```
mysql> SHOW TRIGGERS FROM Fest_database;
+-----+-----+-----+-----+
| Trigger | Event | Table | Statement |
+-----+-----+-----+-----+
| prevent_bulk_purchase | INSERT | purchased | BEGIN
  IF NEW.quantity > 5 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Cannot purchase more than 5 units of a single item in one transaction';
  END IF;
END | BEFORE | 2025-10-03 17:36:55.60 | ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION | root@localhost | cp850 | cp850_general_ci | utf8mb4_0900_ai_ci |
| decrease_item_quantity | INSERT | purchased | BEGIN
  UPDATE stall_items
  SET total_quantity = total_quantity - NEW.quantity
  WHERE stall_id = NEW.stall_id AND item_name = NEW.item_name;
END | AFTER | 2025-10-03 14:47:36.53 | ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION | root@localhost | cp850 | cp850_general_ci | utf8mb4_0900_ai_ci |
```

```

MySQL 8.0 Command Line Cli x + v - □ ×
-----+-----+-----+-----+-----+
| prevent_bulk_purchase | INSERT | purchased | BEGIN
  IF NEW.quantity > 5 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Cannot purchase more than 5 units of a si
nle item in one transaction';
  END IF;
END | BEFORE | 2025-10-03 17:36:55.60 | ONLY_FULL_GROUP_BY,STRICT_TRANS_TABL
ES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTIT
UTION | root@localhost | cp850 | cp850_general_ci | utf8m
b4_0900_ai_ci |
| decrease_item_quantity | INSERT | purchased | BEGIN
  UPDATE stall_items
  SET total_quantity = total_quantity - NEW.quantity
  WHERE stall_id = NEW.stall_id AND item_name = NEW.item_name;
END | AFTER | 2025-10-03 14:47:36.53 |
ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_F
OR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION | root@localhost | cp850
| cp850_general_ci | utf8mb4_0900_ai_ci |
| decrease_quantity_after_purchase | INSERT | purchased | BEGIN
  UPDATE stall_items
  SET total_quantity = total_quantity - NEW.quantity
  WHERE stall_id = NEW.stall_id
  AND item_name = NEW.item_name;
END | AFTER | 2025-10-03 17:33:54.01 | ONLY
_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DI
VISION_BY_ZERO,NO_ENGINE_SUBSTITUTION | root@localhost | cp850
| cp850_general_ci | utf8mb4_0900_ai_ci |
-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)

mysql>

```

-- Show all procedures

SHOW PROCEDURE STATUS WHERE Db = 'Fest_database';

```

mysql> SHOW PROCEDURE STATUS WHERE Db = 'Fest_database';
-----+-----+-----+-----+-----+
| Db      | Name                | Type      | Definer      | Modifie
d         | Created            | Security_type | Comment      | character_set
_client | collation_connection | Database Collation |
-----+-----+-----+-----+-----+
| fest_database | GetStallSales      | PROCEDURE | root@localhost | 2025-10
-03 18:13:08 | 2025-10-03 18:13:08 | DEFINER      |              | cp850
| cp850_general_ci | utf8mb4_0900_ai_ci |
| fest_database | RegisterParticipant | PROCEDURE | root@localhost | 2025-10
-03 18:24:54 | 2025-10-03 18:24:54 | DEFINER      |              | cp850
| cp850_general_ci | utf8mb4_0900_ai_ci |
-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

```

-- Show all functions

SHOW FUNCTION STATUS WHERE Db = 'Fest_database';

```
MySQL 8.0 Command Line Cli x + v - □ x

mysql> SHOW PROCEDURE STATUS WHERE Db = 'Fest_database';
+-----+-----+-----+-----+-----+
| Db      | Name          | Type      | Definer      | Modified |
|-----|-----|-----|-----|-----|
| fest_database | GetStallSales | PROCEDURE | root@localhost | 2025-10-03 18:13:08 |
| cp850_general_ci | utf8mb4_0900_ai_ci | DEFINER | | cp850 |
| fest_database | RegisterParticipant | PROCEDURE | root@localhost | 2025-10-03 18:24:54 |
| cp850_general_ci | utf8mb4_0900_ai_ci | DEFINER | | cp850 |
+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> SHOW FUNCTION STATUS WHERE Db = 'Fest_database';
+-----+-----+-----+-----+-----+
| Db      | Name          | Type      | Definer      | Modified |
|-----|-----|-----|-----|-----|
| fest_database | GetEventCost | FUNCTION | root@localhost | 2025-10-03 18:26:50 |
| cp850_general_ci | utf8mb4_0900_ai_ci | DEFINER | | cp850 |
| fest_database | GetParticipantPurchaseTotal | FUNCTION | root@localhost | 2025-10-03 18:28:43 |
| cp850_general_ci | utf8mb4_0900_ai_ci | DEFINER | | cp850 |
+-----+-----+-----+-----+-----+
```